

Mastering Processes in Linux: A Comprehensive Tutorial

As a Linux user, you must know how to handle processes. By understanding how to start, list, and kill processes, you can effectively manage resources and ensure smooth operations. This tutorial discusses a few commands that will help you gain valuable insights into the processes running on your Linux system.



A process is an instance of a program executing on a system, and Linux provides different ways for managing processes. In this tutorial, we will explore the basics of handling processes in Linux, including starting processes, listing running processes, and killing processes. We will also discuss the concept of parent and child processes. Understanding how to handle processes is an essential skill for any Linux user.

Starting processes

A process is a running instance (active) of a program (passive). There are several ways to start a process in Linux. You can initiate a process in Linux by using the command line, shell scripts, or a GUI. By default, a process runs with the permissions of the user running it.

When a process is started, Linux assigns it a unique identifier known as a process ID (PID). This is a numerical value, which is used by the system to track and manage the processes running on it. PIDs are integers, and their range may vary depending on the kernel version. PID 0 is reserved for the swapper or scheduler process, while PID 1 is reserved for the init process, which is the first process to start when the system boots up.

Listing running processes

To list all running processes on your system, you can use the 'ps' command ('process status'). It displays information about the processes running on your current terminal session.

```
$ps
```

You can use various options with the 'ps' command to display information about all the processes running on your system and their PIDs. The '-ef' option displays information about all the processes running on your system in a detailed format, including the process ID (PID), the parent process ID (PPID), the user who started the process, the amount of CPU and memory the process is using, and the command that started the process.

```
$ps -ef
```

Another useful option is the 'aux' option, which displays information about all the processes running on your system,

```
[sn@localhost ~]$ ps
  PID TTY          TIME CMD
 3964980 pts/0    00:00:00 bash
 3965317 pts/0    00:04:01 job1
 3965370 pts/0    00:00:00 ps
[sn@localhost ~]$
```

Figure 1: Output of the 'ps' command